

Building libterae

The Makefile, from a desktop to a bare board

Emilio Schininà, M.Sc. in Automation Engineering
Version 0.3
November 8, 2026

Copyright © 2026 Emilio Schininà, M.Sc. in Automation Engineering. libterae is licensed under the GNU General Public License, version 3 or any later version, or under a commercial license.

building-en.tex Page ii November 8, 2026 13:36

Contents

1 The Build **1**
1.1 In five lines . . . 1
1.2 What it needs . . . 1
1.3 What it makes . . . 2
1.4 What a program links . . . 2

2 The Variables **3**
2.1 The archiver . . . 4
2.2 The page size . . . 4
2.3 The sizes of the tables . . . 5
2.4 The build folder . . . 6

3 Choosing the System **7**
3.1 Another system, from this machine . . . 8

4 The Memory Functions **9**
4.1 The folders . . . 9
4.2 How `PORT=auto` decides . . . 10
4.3 The memory archive alone . . . 11

5 Building with TLS **13**
5.1 Mbed TLS . . . 13
5.2 The library and the program . . . 14
5.3 TLS on a board . . . 14

6 The Shared Library **15**
6.1 What it makes . . . 15
6.2 What a program links . . . 16
6.3 The version . . . 16
6.4 What it gives a program . . . 17
6.5 With TLS . . . 17

building-en.tex Page iv November 8, 2026 13:36

6.6 Where there is none 18

7 Embedded Systems **19**
7.1 A bare build 19
7.2 What the program provides. 20
7.3 A real-time kernel 21
7.4 What a board gives 21
7.5 Flags for a firmware 22

8 Another C Library **23**
8.1 What `platform/posix/` needs 23
8.2 Pointing the compiler at it 24
8.3 Checking what a build calls 24

9 Reference **25**
9.1 The targets 25
9.2 When the build stops. 25
9.3 libterae in another project's build 27

1

The Build

The library is built by the `Makefile` at the root of the repository, with GNU `make` and a C99 compiler. There is no configuration step: the command line is the whole configuration, and the archives written to a folder of `build/` are the whole result, with a shared library in `lib/` when it is asked for (Chapter 6).

1.1 In five lines

```
# This machine: build/libterae.a
make
# Another system: build/win32/libterae.a
make OS=win32 CC='zig cc -target x86_64-windows-gnu' AR='zig ar' PAGE_SIZE=4096
# A board with no system and no C library: build/none-armv7m/, two archives
make OS=none PORT=auto CROSS_COMPILE=arm-none-eabi- \
     CFLAGS='-mcpu=cortex-m4 -mthumb -O2' PAGE_SIZE=4096
# With TLS: build/tls/libterae.a
make WITH_TLS=yes MBEDTLS_PREFIX="$HOME/mbedtls"
# The commands of a build, printed rather than run, every one of them
make -n -B OS=posix
```

1.2 What it needs

- **A C99 compiler.** `cc`, unless `CC` names another: a cross compiler, or a wrapper with its arguments, such as `zig cc -target x86_64-linux-musl`.
- **GNU `make`.** Version 3.81, the one macOS installs, or later. BSD `make` cannot read the file.
- **An archiver for the target.** Found from `CC`, or named by `AR`, as §2.1 explains.

Nothing else: no CMake, no pkg-config, no script. Mbed TLS is needed only for TLS, and its own build uses CMake (Chapter 5).

building-en.tex Page 2 November 8, 2026 13:36

1.3 What it makes

- **`libterae.a`.** The library: `src/`, with the packets, the topics, the Server and the Client, and, unless `OS=none`, the files of `platform/` that join them to a system's sockets, clock, TLS, and the files or flash a store of Session state is kept in.
- **`libterae_port.a`.** `memcpy`, `memmove`, `memset` and `memcmp`, for a build with no C library to give them. It is made only when `PORT` names a folder of `src/port/` (Chapter 4).
- **A shared library.** With `make shared`, the library as one a program loads when it starts, in `lib/`, and `pkgconfig/libterae.pc` beside it (Chapter 6).

Neither archive has a `main`, a thread or a call to an allocator: every byte `src/` uses is given to it by the program. `platform/` has static tables, which the numbers of Section 2.3 size, and a buffer of about 128 KiB that a store is restored and copied through.

1.4 What a program links

```
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U main.c build/libterae.a -o program
```

The program reads the headers of `include/`, and is compiled with the page size the library was built with: `MQTT_PAGE_SIZE` sets the size of arrays the program hands to the library, and the archive does not record it (§2.2). Whatever else a build needs comes after `libterae.a`: `libterae_port.a` for a build with `PORT`, the three archives of Mbed TLS for one with TLS, and `ws2_32` and `bcrypt` on Windows.

2

The Variables

Every choice is a variable on the command line, as in `make OS=posix`. `make` also takes them from the environment, so one can be set once for a shell.

Table 2.1 The variables of the build

Variable	What it chooses
`OS`	The folder of `platform/`: `posix`, `linux`, `macos`, `win32`, `freertos`, `ucos3`, or `none` for `src/` alone. By default this machine's: `macos` on Darwin, `linux` on Linux, `posix` elsewhere (Chapter 3).
`PORT`	A folder of `src/port/`, for a build with no C library, or `auto` for the one that fits the processor. Empty by default, which leaves the memory functions to the C library (Chapter 4).
`CROSS_COMPILE`	The prefix of a GNU toolchain, as `arm-none-eabi-`. `CC` becomes that prefix followed by `gcc`, unless `CC` is given too.
`CC`	The compiler, with any arguments it needs. `cc` by default.
`AR`	The archiver (§2.1).
`CFLAGS`	The optimisation, the processor and the debugging information. `-O2 -g` by default.
`STRICT`	The language and the warnings of `src/`: `-std=c99 -pedantic -Wall -Wextra -Werror -Wshadow -Wconversion -Wno-sign-conversion`. `STRICT=` clears them all, for a compiler that refuses one.
`WITH_TLS`	`yes` builds `platform/platform_tls.c` with Mbed TLS. `no`, the default, builds it with every function reporting `MQTT_TLS_ERROR` (Chapter 5).

continues

building-en.tex Page 4 November 8, 2026 13:36

Table 2.1 The variables of the build (*continued*)

Variable	What it chooses
MBEDTLS_PREFIX	The folder Mbed TLS is installed in, when the compiler does not find its headers by itself.
BUILD	The folder of the objects and the archives (§2.4).
PAGE_SIZE	The page size, a power of two from 4096 to 65536 (§2.2).
RTOS_INCLUDES	The -I options of a real-time kernel, its network stack and the program's configuration, for freertos and ucos3 (§7.3).
PLATFORM_DEFS	Definitions given to the files of platform/ alone.
WAIT_SOCKETS SOCKETS NET_EVENTS TLS_SESSIONS TLS_POOL_BYTES	The sizes of the tables in platform/ (§2.3).

2.1 The archiver

AR defaults to the llvm-ar beside CC, if CC names one that exists; otherwise to the ar that CC names when asked with -print-prog-name=ar; otherwise to ar. A GNU toolchain names its own, and so needs nothing. A compiler that names none, such as zig cc or a clang with no llvm-ar beside it, falls to the ar of this machine, which may write an archive whose index the target's linker cannot read. Give that build an archiver that writes the target's format: AR='zig ar', or an llvm-ar.

2.2 The page size

MQTT_PAGE_SIZE is the unit the Server's arenas are cut into, and it should be the page of the system that runs the program. By default the build takes it from the machine that runs make:

- **16384.** Darwin on arm64, and loongarch64.
- **65536.** Linux on ppc64 and ppc64le.
- **4096.** Everything else.

A cross build is for another machine, so it gives PAGE_SIZE itself: on a Mac with Apple silicon, every build would otherwise get 16384. The program is compiled with the same number, as -DMQTT_PAGE_SIZE; a program compiled without it gets the header's default of 4096. A number out of range, or not a power of two, stops the build

building-en.tex Page 5 November 8, 2026 13:36

in `include/base_page.h`:

```
$ make PAGE_SIZE=3000
include/base_page.h:60:2: error: "MQTT_PAGE_SIZE must be from 4096 to 65536"
```

The page size is not part of the name of the build folder, so a build with another one is given its own: `make PAGE_SIZE=4096 BUILD=build/page4096`.

2.3 The sizes of the tables

Five numbers size the static tables of `platform/`. The defaults suit a desktop server and take megabytes; `freertos` and `ucos3` set smaller ones, and any build can set its own:

Table 2.2 The sizes of the tables

Variable	What it sizes	Default
`WAIT_SOCKETS`	`MQTT_PLATFORM_WAIT_SOCKETS`: the sockets one wait covers, from 8 to 1048576.	65536; 32 on `freertos` and `ucos3`
`SOCKETS`	`MQTT_PLATFORM_SOCKETS`: the table of handles of `win32` and `freertos`, whose sockets are not small numbers.	`WAIT_SOCKETS` plus 64; 48 on `freertos`
`NET_EVENTS`	`MQTT_NET_EVENTS`: the events one wait reports.	1024; 32 on `freertos` and `ucos3`
`TLS_SESSIONS`	`MQTT_NET_TLS_SESSIONS`: the TLS sessions at one time, from 1 to 65536.	1024; 1 on `freertos` and `ucos3`
`TLS_POOL_BYTES`	`MQTT_TLS_POOL_BYTES`: the static pool Mbed TLS takes its memory from.	64 MiB

Each is a definition given to every file the build compiles, and none changes a structure the program sees. A program that reads one of these limits from the headers, as `max_sockets` is checked against `MQTT_PLATFORM_WAIT_SOCKETS`, is compiled with the same definitions. Every compile line has them, so `make -n` shows what a build takes:

```
make -n -B OS=freertos RTOS_INCLUDES=-I. | grep -o -- '-DMQTT_[A-Z_]*=[0-9]*U*' \
  | sort -u
```

building-en.tex Page 6 November 8, 2026 13:36

2.4 The build folder

Two builds that differ in any way must not share objects, so `BUILD` defaults to a folder of `build/` named by what sets the build apart: `tls` when `WITH_TLS=yes`, then the `OS` when it is not this machine's, then the folder `PORT` takes, joined by hyphens.

```
make                                          # build/
make WITH_TLS=yes                             # build/tls/
make OS=win32 CC=...                          # build/win32/
make OS=none PORT=auto CROSS_COMPILE=arm-none-eabi- CFLAGS='-mcpu=cortex-m4 ...'
                                              # build/none-armv7m/
make WITH_TLS=yes OS=freertos PORT=armv7m ... # build/tls-freertos-armv7m/
```

`CC`, `CFLAGS`, `PAGE_SIZE` and the sizes are not in the name, so two Linux builds, one with glibc and one with musl, both go to `build/linux/`. That is safe, since the second rebuilds everything, but to keep both, give each its own `BUILD`.

Inside the folder the build keeps a file named `flags-` and a checksum of the compiler and every flag. When any of them changes, the name changes and everything in the folder is built again, so a new `CFLAGS` needs no `make clean`. A toolchain upgraded in place, under the same name, is not seen: build it once with `make -B`.

`make clean` removes `BUILD`, and the folder of `lib/` that `make shared` would use, which is inside `BUILD` when `BUILD` is given. With no variable, that is all of `build/` and `lib/`; `make clean OS=win32` removes `build/win32/` and `lib/win32/` alone, and a `BUILD` outside the tree is removed where it is. A `LIBDIR` given to `make` is never removed.

3

Choosing the System

The folder of `platform/` that talks to the system is chosen by `OS`. Every system builds the six files at the top of `platform/` that all of them share (`platform_net_server.c`, `platform_net_store.c`, `platform_net_client.c`, `platform_net_runs.c`, `platform_tls.c` and `platform_parse.c`), the shared files for waiting, for handles and for a store that it uses, and its own folder. A store is kept in files, through `platform_store_file.c`, on every system but the two real-time ones, which keep it in flash, through `platform_store_flash.c`. Each folder is compiled with its own definitions, so `posix/` sees POSIX alone, even in a macOS build, and the top of `platform/` includes no system header at all.

Table 3.1 The systems

OS	What it builds
`macos`	`posix/` for the sockets, the clock and files, and a wait with `kqueue` and a sync with `F_FULLFSYNC`, which reaches the drive's own cache, in `macos/`, which is compiled with `-D_POSIX_C_SOURCE=200809L -D_DARWIN_C_SOURCE`.
`linux`	`posix/`, and a wait with `epoll`, in `linux/`.
`posix`	POSIX.1-2008 alone, compiled with `-D_POSIX_C_SOURCE=200809L`, waiting with `poll`, but for two things: random bytes from `/dev/urandom`, a file POSIX doesn't name, and `fsync`, from POSIX's optional File Synchronization. For any other Unix.
`win32`	Winsock 2 and the Windows API, 32-bit and 64-bit, from Windows Vista's API on, waiting with `WSAPoll`. A program links `ws2_32` and `bcrypt`.
`freertos`	FreeRTOS+TCP, waiting with its socket sets. It needs `RTOS_INCLUDES`, and the board's calls of Section 7.4.

continues

building-en.tex Page 8 November 8, 2026 13:36

Table 3.1 The systems (*continued*)

OS	What it builds
`ucos3`	uC/OS-III with uC/TCP-IP, waiting with `NetSock_Sel`. It needs `RTOS_INCLUDES`, and the board's calls of Section 7.4.
`none`	Nothing of `platform/`: `src/` alone, and so no TLS (Chapter 7).

An `OS` that names no folder stops the build before it starts:

```
$ make OS=beos
Makefile:...: *** OS=beos names no folder of platform/: use posix, linux, macos,
win32, freertos or ucos3, or none.  Stop.
```

3.1 Another system, from this machine

A compiler that carries its own target and C library builds for another system with nothing installed beside it. With `zig`:

```
# POSIX alone, with this machine's compiler.
make OS=posix

# Linux on x86-64, with glibc and with musl.
make OS=linux CC='zig cc -target x86_64-linux-gnu'  AR='zig ar' PAGE_SIZE=4096
make OS=linux CC='zig cc -target x86_64-linux-musl' AR='zig ar' PAGE_SIZE=4096

# Windows, 64-bit and 32-bit.
make OS=win32 CC='zig cc -target x86_64-windows-gnu' AR='zig ar' PAGE_SIZE=4096
make OS=win32 CC='zig cc -target x86-windows-gnu'    AR='zig ar' PAGE_SIZE=4096
```

`AR` is given because `zig cc` names no archiver (§2.1), and `PAGE_SIZE` because the page of the machine that builds is not the page of the one that runs (§2.2).

4

The Memory Functions

A compiler emits calls to `memcpy`, `memmove`, `memset` and `memcmp` even for code that calls none of them: a structure assignment, a large initialiser and a loop that copies can all become one. `src/` calls nothing, but with no C library something still has to answer those calls. `PORT` names the folder of `src/port/` that does, and builds it into `libterae_port.a`:

- **The archive.** It defines the four functions, and on Arm the run-time ABI's `__aeabi_mem*` names too. A program links it right after `libterae.a`, so that the library's calls, the program's and the compiler's are all answered by it.
- **The library.** With a folder of assembly, `src/` is compiled with `-DMQTT_PORT_MEMORY`, and copies with the folder's `memcpy`, under its other name `mqtt_port_memcpy`, instead of its own loop of bytes.

4.1 The folders

Table 4.1 gives the folder for each processor. It is the folder `PORT=auto` takes, so it can be asked rather than looked up (§4.2).

Table 4.1 The folders of `src/port/`

Processor	Folder
Cortex-M3, M4, M7, M33 and M55: the M profile with Thumb-2	`armv7m`
ARM7TDMI, ARM9, ARM11, Cortex-R (TMS570, RM4x), and big-endian Cortex-A	`armv4t`
32-bit Cortex-A, little-endian, with unaligned loads	`arm`

continues

building-en.tex Page 10 November 8, 2026 13:36

Table 4.1 The folders of `src/port/` (*continued*)

Processor	Folder
AArch64 with Advanced SIMD	`aarch64`
x86-64, but not x32; and i386	`x86_64, i386`
RISC-V, 32-bit (ESP32-C3 and C6, SHAKTI E); 64-bit with `M`	`riscv32, riscv64`
MIPS32 with the o32 ABI, not microMIPS (PIC32)	`mips`
64-bit PowerPC with ELFv2	`powerpc64`
AVR with `MOVW`, not AVRtiny	`avr`
SuperH from SH-2, RX with string instructions, 68000 and ColdFire, and Xtensa (ESP32, S2, S3). Assembled or built, but not yet run	`sh, rx, m68k, xtensa`
Anything else: Cortex-M0, M0+ and M23, MSP430, AArch64 with `-mstrict-align`, and every target that is not ELF, such as Windows and macOS	`generic`

`generic` is the four functions in C, one byte at a time, which any processor runs. It is compiled with `-ffreestanding -fno-builtin`, and, with GCC, `-fno-tree-loop-distribute-patterns`, so that no loop is turned into a call to the function it is in. It does not define `MQTT_PORT_MEMORY`, because its `memcpy` is the same loop `src/` would run anyway.

A folder that does not fit the processor stops the build in its own assembly, with an error that says what the folder needs: for `PORT=armv7m` on a Cortex-M0, an M-profile Arm with Thumb-2. A name that is not a folder stops the build and lists the sixteen that are:

```
$ make PORT=sparc
Makefile:...: *** PORT=sparc names no folder of src/port/: use auto, or one of
aarch64 arm armv4t armv7m avr generic i386 m68k mips powerpc64 riscv32 riscv64
rx sh x86_64 xtensa.  Stop.
```

4.2 How `PORT=auto` decides

`PORT=auto` runs the build's own `CC`, with its `CFLAGS`, over `src/port/select.in`: a list of tests on the macros the compiler defines, from which the preprocessor leaves the name of one folder. It must therefore see the compiler and the flags of the real build, and `make print-port` shows what it picks without building anything:

building-en.tex Page 11 November 8, 2026 13:36


```
$ make print-port PORT=auto CROSS_COMPILE=arm-none-eabi- \
      CFLAGS='-mcpu=cortex-m4 -mthumb'
armv7m
$ make print-port PORT=auto CROSS_COMPILE=arm-none-eabi- \
      CFLAGS='-mcpu=cortex-r4f -mbig-endian'
armv4t
$ make print-port PORT=auto CROSS_COMPILE=arm-none-eabi- \
      CFLAGS='-mcpu=cortex-m0 -mthumb'
generic
```

The tests are made in this order, and when `auto` does not take the folder expected, the reason is one of them:

- **Not ELF.** Windows, macOS and every other target that is not ELF take `generic`: every folder of assembly is ELF.
- **AArch64.** Needs `__ARM_NEON` and `__ARM_FEATURE_UNALIGNED`, so `-mstrict-align` gives `generic`.
- **x86.** `x86_64` needs 64-bit pointers, so x32 gives `generic`; `i386` follows.
- **Arm.** The M profile with Thumb-2 gives `armv7m`; the A profile from Armv7, little-endian with unaligned loads, gives `arm`; any other Arm with both the Arm and Thumb instruction sets gives `armv4t`.
- **RISC-V.** Split by `__riscv_xlen`; 64-bit needs `__riscv_mul`.
- **The rest.** MIPS needs o32 and not microMIPS, PowerPC needs 64-bit ELFv2, AVR needs `MOVW` and not AVRtiny, SuperH must not be SH-1 or SH-5, and RX must allow string instructions. Then come `m68k` and `xtensa`; anything left is `generic`.

When the compiler is not on the `PATH`, or refuses a flag of `CFLAGS`, as GCC refuses `-mcpu=cortex-r5f`, which it spells `cortex-r5`, the build stops: `PORT=auto couldn't read src/port/select.in`. Running the compiler by hand on that file, with `-E -P -x c` and the same flags, shows its own message.

4.3 The memory archive alone

A project with a build or a C library of its own may want only `libterae_port.a`. `make port` builds that and nothing else, takes every variable a build does, and stops if `PORT` is not given:

```
make port PORT=aarch64 CC='zig cc -target aarch64-linux-musl' AR='zig ar'
make port PORT=armv7m CROSS_COMPILE=arm-none-eabi- \
     CFLAGS='-mcpu=cortex-m4 -mthumb -O2' BUILD=/path/to/firmware/terae
```

building-en.tex Page 12 November 8, 2026 13:36

5

Building with TLS

TLS is one file, `platform/platform_tls.c`, built against Mbed TLS 3.6 when `WITH_TLS=yes`. Without it the same file is still built, `mqtt_tls_available` returns false, and every other function of `base_tls.h` reports `MQTT_TLS_ERROR`, so a program asks once and decides.

5.1 Mbed TLS

Mbed TLS is not part of the repository; 3.6.7 is the version tested. It is built once with libterae's configuration, `platform/mbedtls_config.h`, named by its absolute path, since Mbed TLS has a file of the same name that a relative one would find first:

```
URL=https://github.com/Mbed-TLS/mbedtls/releases/download
curl -LO "$URL/mbedtls-3.6.7/mbedtls-3.6.7.tar.bz2"
tar -xjf mbedtls-3.6.7.tar.bz2
cmake -S mbedtls-3.6.7 -B mbedtls-build -DCMAKE_INSTALL_PREFIX="$HOME/mbedtls" \
      -DMBEDTLS_USER_CONFIG_FILE=/path/to/libterae/platform/mbedtls_config.h \
      -DENABLE_PROGRAMS=OFF -DENABLE_TESTING=OFF -DMBEDTLS_FATAL_WARNINGS=OFF
cmake --build mbedtls-build && cmake --install mbedtls-build
```

The configuration makes Mbed TLS take its memory from a static pool in `platform_tls.c` rather than from `malloc`, so TLS keeps the library's promise of no allocator. The two settings that do it are `MBEDTLS_PLATFORM_MEMORY` and `MBEDTLS_MEMORY_BUFFER_ALLOC_C`.

building-en.tex Page 14 November 8, 2026 13:36

5.2 The library and the program

```
make WITH_TLS=yes MBEDTLS_PREFIX="$HOME/mbedtls"      # build/tls/libterae.a

cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U main.c build/tls/libterae.a \
   "$HOME/mbedtls/lib/libmbedtls.a" "$HOME/mbedtls/lib/libmbedx509.a" \
   "$HOME/mbedtls/lib/libmbedcrypto.a" -o program
```

The files of `platform/` are then compiled with `-DMQTT_WITH_TLS`, with Mbed TLS's headers from `MBEDTLS_PREFIX`, which can be left out when the compiler finds them by itself, and with the absolute path of `platform/mbedtls_config.h` as Mbed TLS's configuration. The program needs no definition for TLS: only `platform_tls.c` reads one. It links `libterae.a` first, then the three archives of Mbed TLS, each after the one that calls it.

5.3 TLS on a board

A board builds Mbed TLS for its own processor, with the same configuration, and gives the pool a size its RAM can hold. The pool holds a whole Mbed TLS context, with its buffers, for each session, so the two numbers are chosen together:

```
make WITH_TLS=yes MBEDTLS_PREFIX="$HOME/mbedtls-cortex-m4" \
     OS=freertos PORT=auto CROSS_COMPILE=arm-none-eabi- \
     CFLAGS='-mcpu=cortex-m4 -mthumb -O2' RTOS_INCLUDES='-I...' PAGE_SIZE=4096 \
     TLS_SESSIONS=2 TLS_POOL_BYTES=131072      # build/tls-freertos-armv7m/
```

6

The Shared Library

A program can also take libterae from a shared library, which the system's loader finds when the program starts. `make shared` builds one in `lib/`, a folder the repository does not keep. The archive is still what a firmware links, and the tests use; the shared library is for a program on Linux, macOS, Windows or another POSIX system that another project builds, that calls the library from another language, or that should take a new version of it without being linked again.

6.1 What it makes

```
make shared                                          # lib/
make shared WITH_TLS=yes MBEDTLS_PREFIX="$HOME/mbedtls"   # lib/tls/
make shared OS=win32 CC='zig cc -target x86_64-windows-gnu' PAGE_SIZE=4096
                                                     # lib/win32/
```

Table 6.1 The shared library, for version 1.0.5

System	File	Beside it
`linux`, `posix`	`libterae.so.1.0.5`	`libterae.so.1`, the name a program records, and `libterae.so`, which `-lterae` finds: both link to it.
`macos`	`libterae.1.dylib`	`libterae.dylib`, which links to it. A program records `@rpath/libterae.1.dylib`.
`win32`	`libterae-1.dll`	`terae.lib`, the import library, which `-lterae` finds with GNU `ld`, `lld` and `zig`, and Microsoft's linker by name.

building-en.tex Page 16 November 8, 2026 13:36

Each folder also has `pkgconfig/libterae.pc` (§6.2). The objects are compiled again for it, as code that runs at any address, with `-fPIC`, which Windows does not need. They go to a folder `shared` inside `BUILD`, so `libterae.a` stays as it was.

The folder of `lib/` is named as the build's folder of `build/` is (§2.4): `lib/tls`, `lib/win32`, `lib/tls-linux`. When `BUILD` is given, the library goes to a folder `lib` inside it, so that a build with another page size cannot take the place of the default one. `LIBDIR` names any other folder. `make clean` removes the library with its build, but never a `LIBDIR` given to `make`, which may hold other libraries.

6.2 What a program links

```
export PKG_CONFIG_PATH="$PWD/lib/pkgconfig"
cc -std=c99 $(pkg-config --cflags libterae) main.c $(pkg-config --libs libterae) \
   -o program
```

`libterae.pc` records what the program needs from the build: the version, `-I` for `include/`, the page size and the sizes of the tables the library was built with, and `-L` and `-lterae`. On Linux and macOS it also gives the program a run-time path to the folder, so that the loader finds the library there, as it looks in no folder of the repository by itself. Without `pkg-config`, the same line is:

```
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U main.c -Llib -lterae \
   -Wl,-rpath,"$PWD/lib" -o program
```

The program links nothing else: the shared library is already linked with Mbed TLS, in a build with TLS, and with `ws2_32` and `bcrypt` on Windows. On Windows the DLL goes beside the program, or in a folder of the `PATH`, as Windows has no run-time path.

6.3 The version

The library has one version, three numbers from 0 to 255, in `include/base_library.h`: `MQTT_LIBRARY_VERSION_MAJOR`, `_MINOR` and `_PATCH`. The Makefile reads them from there, so the names of the shared library cannot disagree with the headers a program is compiled with, and `make print-version` prints them, as `1.0.5`.

- **Major.** Changes when a program built for the version before may need changing, or building again. The name a program records, `libterae.so.1`,

building-en.tex Page 17 November 8, 2026 13:36

`libterae.1.dylib` or `libterae-1.dll`, changes with it, so a program never loads a library of another major version.

- **Minor.** Changes when the library gains something, and a program built for an earlier minor version still works with it. macOS records it as the library's compatibility version, and its loader refuses a library older than the one a program was linked with.
- **Patch.** Changes when nothing a program can see changes.

The page size is not in the name, yet it is part of what a program and the library share (§2.2). A program that may meet a library built elsewhere asks at the start, with the numbers of its own headers:

```
if (!mqtt_library_compatible(MQTT_LIBRARY_VERSION, MQTT_PAGE_SIZE)) {
    /* Another major version, a later minor one, or another page size. */
    return 1;
}
```

6.4 What it gives a program

A program sees the functions whose names start with `mqtt`, which are all those of `include/`. The rest of `platform/`, and Mbed TLS, stay inside: an ELF system binds a call to the first function of that name it finds, so a function of the program called `platform_wait_call` would otherwise take the place of the library's own. On ELF systems the build writes a list for the linker, `exports.map` in the folder `shared` of `BUILD`, which GNU `ld`, `gold`, `lld` and `mold` take. On macOS a pattern on the command line does the same. A DLL lists every function it has, which is harmless: Windows binds each call to the DLL it names.

6.5 With TLS

The shared library is linked with `-lmbedtls -lmbedx509 -lmbedcrypto` from `MBEDTLS_PREFIX/lib`, so it must be the Mbed TLS built with `platform/mbedtls_config.h` (Chapter 5). Its archives then go inside the shared library. On Linux and the other ELF systems they have to be position-independent too: add `-DCMAKE_POSITION_INDEPENDENT_CODE=ON` to the `cmake` line that configures Mbed TLS. macOS and Windows need nothing.

building-en.tex Page 18 November 8, 2026 13:36

6.6 Where there is none

`make shared` refuses a system with no loader, `OS=freertos`, `ucos3` or `none`, as a firmware links `libterae.a`. It also refuses `PORT`: its `memcpy` and `memset` are for a program with no C library, and a shared library is loaded beside one, whose functions they would take the place of for the whole program.

7

Embedded Systems

On a board, the build takes one of three shapes:

Table 7.1 The shapes of an embedded build

Shape	What is built, and what the program does
Bare, `OS=none`	• `src/` alone, and the memory functions of `PORT`. • The program moves the bytes and gives the time, through `base_server.h` and `base_client.h`: the `*_receive_space` and `*_receive_commit` calls take what arrived, `*_transmit_view` and `*_transmit_consume` give what to send, and `*_tick` runs the timers.
A kernel, `OS=freertos` or `OS=ucos3`	• `src/` and `platform/`, over the kernel's own network stack. • The program calls `mqtt_net_server_run` or `mqtt_net_client_step` from one task, and defines the board's calls of Section 7.4: the output `platform/` prints through, random bytes, and the flash a store is kept in.
A POSIX layer, `OS=posix`	The same as a desktop, on a small system that has POSIX, or on Linux with pages of 4 KiB.

7.1 A bare build

With a GNU toolchain, the archives and then the program:

building-en.tex Page 20 November 8, 2026 13:36


```
make OS=none PORT=auto CROSS_COMPILE=arm-none-eabi- \
     CFLAGS='-mcpu=cortex-m4 -mthumb -O2' PAGE_SIZE=4096
                    # build/none-armv7m/: libterae.a and libterae_port.a

arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -Os -nostdlib -Wl,--gc-sections \
     -T link.ld -Iinclude -DMQTT_PAGE_SIZE=4096U startup.c main.c \
     build/none-armv7m/libterae.a build/none-armv7m/libterae_port.a -lgcc \
     -o firmware.elf
```

`CFLAGS` names the processor, because a toolchain's default is not the board's. `link.ld` and `startup.c` are the board's. The order of the archives is the order a static link needs: the library, then the memory functions it calls, then `-lgcc`, which gives the compiler's own helpers, such as 64-bit division.

With `clang`, the target is part of `CC`, and three things are given by hand: an archiver that writes ELF, `lld` to link, and the compiler's helpers, which Apple's `clang` has none of for Arm. GNU's `libgcc` serves, as does a `compiler-rt` built for the target:

```
make OS=none PORT=auto CC='clang --target=thumbv7m-none-eabi' AR=llvm-ar \
     CFLAGS='-mcpu=cortex-m4 -O2' PAGE_SIZE=4096

clang --target=thumbv7m-none-eabi -mcpu=cortex-m4 -O2 -nostdlib -fuse-ld=lld \
      -Wl,--gc-sections -T link.ld -Iinclude -DMQTT_PAGE_SIZE=4096U \
      startup.c main.c build/none-armv7m/libterae.a \
      build/none-armv7m/libterae_port.a \
      "$(arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -print-libgcc-file-name)" \
      -o firmware.elf
```

Without `-fuse-ld=lld`, `clang` on macOS hands the Arm objects to the system's linker, which answers `ld: unknown options: -Bstatic -EL`.

7.2 What the program provides

- **Memory.** `src/` allocates nothing. The Server takes every array it uses from the `mqtt_server_storage` given to `mqtt_server_init`, sized from `MQTT_PAGE_SIZE` and the counts in `base_server.h`, and the Client likewise.
- **Time.** Every call that needs the time takes `now_ms`, a 64-bit count of milliseconds that only goes forward; a board's 32-bit tick counter is extended by the program.
- **Output.** The library prints nothing, and `platform/` prints only through the board's `*_board_write`.
- **Flash.** A store of Session state is kept in a region of flash the board names, on a real-time kernel, or by the program itself with `OS=none`.

building-en.tex Page 21 November 8, 2026 13:36

7.3 A real-time kernel

`OS=freertos` and `OS=ucos3` need the kernel's headers, the network stack's and the program's configuration, all in `RTOS_INCLUDES`, which only the files of that system's folder are compiled with. The kernel itself is the program's to build and link:

```
make OS=freertos PORT=auto CROSS_COMPILE=arm-none-eabi- PAGE_SIZE=4096 \
     CFLAGS='-mcpu=cortex-m3 -mthumb -O2 -ffunction-sections -fdata-sections' \
     RTOS_INCLUDES="-I$FREERTOS/include -I$FREERTOS/portable/GCC/ARM_CM3 \
                    -I$PLUS_TCP/source/include \
                    -I$PLUS_TCP/source/portable/Compiler/GCC -I$BOARD"
```

The build goes to `build/freertos-armv7m/`, and the folder named by `$BOARD` holds `FreeRTOSConfig.h` and `FreeRTOSIPConfig.h`. uC/OS-III is built the same way with `OS=ucos3`. Both systems already set the small sizes of Table 2.2; a board that wants others gives them on the command line, and to its program.

7.4 What a board gives

On `OS=freertos` and `OS=ucos3`, what the kernel doesn't have comes from the program's board, as functions the program defines, declared in `platform/freertos/freertos_internal.h` and `platform/ucos3/ucos3_internal.h`. A program that links the loops of `platform/` needs them all, as the loops reach the code of the store, which calls them.

- **Output.** `freertos_board_write` or `ucos3_board_write`: the console, for what `platform/` prints.
- **Random bytes.** `xApplicationGetRandomNumber`, which FreeRTOS+TCP asks of every program anyway, or `ucos3_board_random`.
- **Flash for a store.** The seven calls of Table 7.2, each starting `freertos_board_store_` or `ucos3_board_store_`, and returning true when it did what was asked. A board with no flash for a store defines them too, and has `_region` return false, so that no store opens.

building-en.tex Page 22 November 8, 2026 13:36

Table 7.2 The board's calls for flash

Call	What it does
`_region`	Finds the region a store of a name is kept in: its number, its size, a whole number of erase blocks, and the size of a block.
`_read`	Reads bytes of a region.
`_program`	Programs a run of bytes that are erased, whatever the flash's own unit of programming.
`_erase`	Erases whole blocks of a region.
`_sync`	Makes what was programmed outlast a loss of power, for a flash, or a controller in front of it, that holds writes back.
`_set_aside`	Keeps what a region holds somewhere else, under a number of its own, before the store erases it, or returns false if the board has nowhere to keep it: nothing is lost.
`_remove`	Removes a region and what was set aside from it, for the tests.

The flash is taken to be as NOR flash is: erasing sets every byte of a block to 0xFF, and programming only clears bits of bytes that are erased.

7.5 Flags for a firmware

Table 7.3 Flags for a firmware

Flag	What it does
`-ffunction-sections` `-fdata-sections` `-Wl,--gc-sections` at the link	Every function and datum in a section of its own, and only those the firmware reaches are kept.
`-fno-unwind-tables` `-fno-asynchronous-unwind-tables`	No tables for unwinding the stack, which a C firmware never reads.
`-nostdlib` and `-lgcc`	No start-up files or C library that the board does not have, and the compiler's helpers, which it does need.
The processor's: `-mcpu`, `-mthumb`, `-mfloat-abi`, `-mfpu`	The same for the library, its port folder and the program, or their calls do not agree.

8

Another C Library

The library asks little of a C library. `src/` includes only `stddef.h`, `stdint.h` and `stdbool.h`, which every compiler has. `platform/posix/` calls only what POSIX.1-2008 has, and is compiled with `_POSIX_C_SOURCE` alone, so a C library with those functions builds it. Two things go past POSIX.1-2008's base: `fsync`, from its optional File Synchronization, and the file `/dev/urandom`, which it doesn't name.

8.1 What `platform/posix/` needs

- **Headers.** `errno.h`, `fcntl.h`, `limits.h`, `netdb.h`, `netinet/in.h`, `netinet/tcp.h`, `poll.h`, `signal.h`, `string.h`, `sys/socket.h`, `sys/stat.h`, `time.h` and `unistd.h`.
- **Sockets.** `socket`, `bind`, `listen`, `accept`, `connect`, `send`, `sendmsg`, `recvmsg`, `setsockopt`, `socketpair`, `fcntl`, `close`, `getaddrinfo` and `freeaddrinfo`.
- **Waiting, time and signals.** `poll`, `pipe`, `read`, `write`, `clock_gettime`, `sysconf`, `sigaction` and `sigemptyset`.
- **Files.** `open`, `pread`, `pwrite`, `ftruncate`, `fstat`, `lstat`, `mkdir`, `link`, `unlink`, `rmdir`, `fsync` and `time`, for a store kept in files, and `open` for `/dev/urandom` too.
- **Strings.** `memcpy`, `memset` and `strlen`.
- **Processes.** `fork`, `alarm`, `clock` and `_exit`, behind `mqtt_platform_fork`, `mqtt_platform_interrupt_after_s` and `mqtt_platform_cpu_us`. They are in a file of their own, so a static link takes them only into a program that calls those functions.

No `malloc`, no `printf` and no threads.

building-en.tex Page 24 November 8, 2026 13:36

8.2 Pointing the compiler at it

The C library is a matter of `CC`, which can carry any arguments: a sysroot for a whole C library, or include paths for its headers alone.

```
# A complete C library, here musl's, through a compiler that carries it.
make OS=posix CC='zig cc -target x86_64-linux-musl' AR='zig ar' PAGE_SIZE=4096

# A C library's headers, with no other but the compiler's own.
make OS=posix PAGE_SIZE=4096 \
     CC="cc -nostdinc -isystem $HOME/my-libc/include \
         -isystem $(cc -print-file-name=include)"
```

`-nostdinc` hides every other C library, and the second `-isystem` puts back the compiler's own headers, such as `stdint.h`. `-isystem` rather than `-I` keeps the C library's warnings out of a build that has `-Werror`.

A project with a C library of its own either has the four memory functions, and leaves `PORT` out, or takes them from `make port` (Chapter 4), and links `libterae_port.a` before its C library, so that its definitions answer every call.

8.3 Checking what a build calls

What an archive asks of the C library is what its members leave undefined, less what other members define. With `nm`, of macOS or of GNU:

```
nm -u -j build/posix/libterae.a | grep -v -e ':$' -e '^$' | sort -u > needed
nm -g -U -j build/posix/libterae.a | grep -v -e ':$' -e '^$' | sort -u > defined
comm -23 needed defined
```

For an `OS=posix` build, that is the list above, and what the compiler adds of its own: on macOS, `bzero` for a `memset` of zeros, the function behind `errno`, and, because Apple's `clang` turns on the stack protector, `__stack_chk_fail` and `__stack_chk_guard`. A C library without those builds with `-fno-stack-protector` in `CFLAGS`. A new C library is checked by building, then finding no name in the list that it lacks.

9

Reference

9.1 The targets

- **`make`, `make all`.** `libterae.a`, and `libterae_port.a` too when `PORT` is given.
- **`make port`.** `libterae_port.a` alone.
- **`make print-port`.** The folder `PORT` takes, after `auto` has decided; nothing without `PORT`.
- **`make shared`.** The shared library and `libterae.pc`, in `lib/` or the folder of it named by the build (Chapter 6).
- **`make print-version`.** The library's version, as `1.0.5` (§6.3).
- **`make clean`.** Removes `BUILD`, and the folder of `lib/` the build would use, unless `LIBDIR` is given.

9.2 When the build stops

Table 9.1 What a failure means

Message	Cause, and what to do
`OS=... names no folder of platform/`	A misspelt system: the message lists the folders.
`PORT=... names no folder of src/port/`	A misspelt folder, or a processor's name: see Table 4.1.
`PORT=auto: ... couldn't read src/port/select.in`	The compiler is not on the `PATH`, or refused a flag (§4.2).

continues

building-en.tex Page 26 November 8, 2026 13:36

Table 9.1 What a failure means (*continued*)

Message	Cause, and what to do
`OS=none builds no platform/, where TLS is`	TLS is a file of `platform/`: choose a system, or leave TLS out.
`make shared: OS=... has no loader`	A board links `libterae.a`: leave `shared` out.
`make shared: PORT is for a program with no C library`	A shared library runs beside a C library: build it without `PORT`.
`include/base_library.h gives no version`	A definition of `MQTT_LIBRARY_VERSION_MAJOR`, `_MINOR` or `_PATCH` is missing, or is not a number followed by `U`.
`MQTT_PAGE_SIZE must be from 4096 to 65536`, or `must be a power of two`	`PAGE_SIZE` is wrong, in the build or in the program's `-DMQTT_PAGE_SIZE`.
`MQTT_PLATFORM_WAIT_SOCKETS must be from 8 to 1048576`, or `MQTT_NET_TLS_SESSIONS must be from 1 to 65536`	A size given on the command line is out of range.
`FreeRTOS.h` or `os.h` not found	`RTOS_INCLUDES` is missing, or lacks the kernel, the stack or the board's configuration.
`mbedtls/ssl.h` not found	`WITH_TLS=yes` with no `MBEDTLS_PREFIX`, and Mbed TLS is not where the compiler looks.
An undefined `mqtt_port_memcpy` at the link	The library was built with `PORT`: link `libterae_port.a` right after `libterae.a`.
An undefined `memset` or `memcpy` at a bare link	The library was built without `PORT`, and nothing gives them: build it with `PORT`, or link a C library.
`ranlib: warning: ... not a mach-o file`, then an undefined symbol that the archive has	This machine's archiver wrote an archive for another target, with no index: name the target's in `AR`.
`ld: unknown options: -Bstatic -EL`	`clang` gave Arm objects to the system's linker: add `-fuse-ld=lld`.
`undefined symbol: __aeabi_uldivmod`	A bare link without the compiler's helpers: add `-lgcc`, or `compiler-rt`.
`relocation R_X86_64_32 ... recompile with -fPIC` in `make shared WITH_TLS=yes`	Mbed TLS's archives are not position-independent: build them again with `-DCMAKE_POSITION_INDEPENDENT_CODE=ON`.

continues

building-en.tex Page 27 November 8, 2026 13:36

Table 9.1 What a failure means (*continued*)

Message	Cause, and what to do
`Library not loaded: @rpath/libterae.1.dylib`, or `libterae.so.1: cannot open shared object file`	The program has no run-time path to `lib/`: link it with `-Wl,-rpath,<folder>`, as `libterae.pc` does.

9.3 libterae in another project's build

A project that ships libterae in a firmware builds it with every variable written down, so that the build is the same on any machine with the toolchain:

```
TERAE_BUILD := $(abspath build/terae)

$(TERAE_BUILD)/libterae.a: FORCE
        $(MAKE) -C libterae BUILD=$(TERAE_BUILD) OS=freertos PORT=armv7m \
                CROSS_COMPILE=arm-none-eabi- CFLAGS="$(BOARD_CFLAGS)" \
                RTOS_INCLUDES="$(BOARD_INCLUDES)" PAGE_SIZE=4096
FORCE:
```

`BUILD` is absolute, as `make -C` changes folder first. The rule always runs, and the inner `make` decides what is out of date. The program is compiled with the same `-DMQTT_PAGE_SIZE=4096U`. A project for a desktop that wants the shared library asks for `shared` the same way, and finds it, with its `libterae.pc`, in `$(TERAE_BUILD)/lib`.